

LightHD: A Lightweight and High-Performance Hardware Accelerator of CRYSTALS-Dilithium

Ziying Ni , *Member, IEEE*, Ayesha Khalid , *Senior Member, IEEE*, Zhaoyu Zhang , Yijun Cui , *Member, IEEE*, Weiqiang Liu , *Senior Member, IEEE*, and Máire O'Neill , *Senior Member, IEEE*

Abstract—CRYSTALS-Dilithium serves as the foundation of the NIST-standardised PQC digital signature scheme, and has been declared as the first recommended digital signature algorithm. However, due to the computational complexity and intricate processing flow of CRYSTALS-Dilithium, two limitations are shown in existing methods: its applicability on resource-constrained devices is limited and the performance reported so far remains relatively low. This paper presents a lightweight yet high-performance hardware architecture that optimises the core computational units of CRYSTALS-Dilithium. First, an iterative dual-Keccak SHA-3 module is proposed, where two cores operate with a 26-cycle offset to accelerate processing without compromising frequency. In addition, the rejection sampler is streamlined by two compact registers for intermediate values and counters, improving efficiency when consuming interleaved SHA-3 outputs. Second, for small bit-width polynomials, we eliminate the first NTT stage via lookup tables and data regrouping, reducing NTT cycles by 11.7% with little hardware overhead. Further hardware savings are achieved by maximising IP core utilisation and simplifying input multiplexers. Furthermore, a compact scheduling strategy ensures that all intermediate storage fits within a single polynomial-sized memory block. On Xilinx Artix-7 FPGAs, the design reduces hardware overhead by 14.2% compared with state-of-the-art lightweight implementations. Across three security levels, **KeyGen** and **Verify** are 27.3% and 13.5% faster, respectively, than high-performance prior designs. At level 5, the best-case **Sign** latency is only 120 μ s.

Index Terms—Post-quantum cryptography, CRYSTALS-Dilithium, Hardware implementation, ML-DSA, FIPS 204.

I. INTRODUCTION

POST quantum cryptography (PQC) refers to public-key cryptography based on complex mathematical problems that are believed to remain secure against quantum adversaries. Deployed public-key schemes such as RSA [1] and elliptic-curve cryptography (ECC) [2] would be efficiently broken by quantum computers due to Shor's algorithm [3], whilst Grover's algorithm [4] reduces the effective security of symmetric primitives and hash functions. In 2016, the US National Institute of Standards and Technology (NIST) initiated the PQC standardisation process, with 69 schemes submissions in the Round 1 [5]. After four rounds, NIST selected five schemes for standardisation, i.e., CRYSTALS-Kyber [6], CRYSTALS-Dilithium [7], SPHINCS+ [8], Falcon [9], and HQC [10]. In August 2024, NIST published released Federal Information Processing Standards (FIPS: *FIPS 203* (ML-KEM) [11], *FIPS 204* (ML-DSA) [12], and *FIPS 205* (SLH-DSA) [13]. The CRYSTALS-Dilithium digital signature scheme, that makes the basis of the FIPS 204, is widely regarded as the primary digital signature algorithm for broad deployment, owing to its membership in the CRYSTALS family of lattice-based schemes, relatively high efficiency and simple implementation, and strong theoretical foundations [14], [15].

NIST evaluates PQC candidates primarily on security and performance, with particular attention to deployability on lightweight platforms such as Internet-of-Things (IoT) devices. Compared with certain instruction-set-based co-processor designs [16], [17], [18], pure-hardware implementations can deliver higher computational efficiency and throughput. For CRYSTALS-Dilithium, prior hardware research falls broadly into two lines: high-speed architectures [19], [20], [21], [22], [23] and lightweight designs [14], [24], [25]. In the context of lightweight architectures, Gupta et al. report one of the most lightweight hardware architecture to date, with the entire CRYSTALS-Dilithium implementation using only 13,975 LUTs and 6,875 FFs on Artix-7 [24], though it consumes 35 BRAMs and runs nearly 4 $\times$ slower than state-of-the-art high-performance architectures. Moreover, all existing architectures operate at relatively low frequencies. For example, the design in [14] and [21] executes at only 96.9 MHz and 119 MHz on Artix-7, respectively, which significantly lags behind other standardized PQC implementations. A common contributing factor is the widespread use of dual-Keccak hashing design in CRYSTALS-Dilithium, where cascading two rounds of the

Received 10 October 2025; revised 4 January 2026; accepted 15 February 2026. Date of publication 23 February 2026; date of current version 9 April 2026. This work was supported in part by the Innovate UK SCHEME Project under Grant IUK10065634, in part by the Engineering and Physical Sciences Research Council (EPSRC) Quantum Communications Hub under Grant EP/T001011/1, in part by NSFC Distinguished Young Scholars Fund under Grant 62425404, in part by the National Natural Science Foundation of China under Grant 62022041, and in part by the Fundamental Research Funds for the Central Universities under Grant NP2022103. Recommended for acceptance by Y. Hu. (*Corresponding authors: Ziying Ni; Weiqiang Liu*)

Ziying Ni, Ayesha Khalid, and Máire O'Neill are with the Centre for Secure Information Technologies (CSIT), Queens University Belfast, BT3 9DT Belfast, U.K. (e-mail: z.ni@qub.ac.uk; a.khalid@qub.ac.uk; m.oneill@ecit.qub.ac.uk).

Zhaoyu Zhang is with the School of EEECS, Queens University Belfast, BT3 9DT Belfast, U.K. (e-mail: zzhang55@qub.ac.uk).

Yijun Cui and Weiqiang Liu are with the College of Integrated Circuits, Nanjing University of Aeronautics and Astronautics, Nanjing 211106, China (e-mail: yijun.cui@nuaa.edu.cn; liuweiqiang@nuaa.edu.cn).

Digital Object Identifier 10.1109/TC.2026.3666457

Keccak permutation can constrain the datapaths and reduce the overall maximum frequency. Despite these efforts, there remains substantial headroom to optimise Dilithium hardware for both speed and area.

To address the slow speed and large hardware overhead of current CRYSTALS-Dilithium hardware architectures, we propose a high-speed and lightweight CRYSTALS-Dilithium architecture. To the best of our knowledge, the proposed CRYSTALS-Dilithium architecture is the fastest and most lightweight FPGA-based hardware accelerator to date. We first adopt a novel iterative Keccak architecture that significantly reduces the overall area consumption while increasing the operating frequency to 212 MHz, effectively realising the performance potential of other components. Additionally, we further optimise key modules, such as the rejection sampler and NTT, to reduce both area and cycle count. A more compact scheduling method is proposed to eliminate the need for additional polynomial multiplication modules and memory. The main contributions of this work are summarised as follows.

- An interleaved SHA-3 operation strategy is proposed to reduce the resource consumption and enhance overall operating frequency. Additionally, a sampler for the interleaved SHA-3 output is introduced, which uses only two small registers to sample two polynomials in real time.
- The calculation overhead of first stage is completely eliminated through regrouping and lookup tables of small bit-width vectors in number theoretic transform (NTT) computation, reducing the cycle count of a single NTT computation by 11.7%. In addition, merging part of the operations with the DSP blocks and reducing the input multiplexing logic is done to further reduce hardware overhead in the polynomial module.
- A more compact scheduling method is proposed, processing polynomial computations iteratively on a row-by-row basis. This approach requires only a single polynomial-sized memory for intermediate values throughout the entire operation, eliminating the need for additional polynomial multiplication modules.
- TeX ?>Implementation results on the Artix-7 FPGA demonstrate that our design is the most lightweight, reducing hardware area by 14.2% compared to the state-of-the-art. In addition, the proposed design speedup **KeyGen** and **Verify** phases by 27.3% and 13.5%, respectively. In the best-case scenario at security level 5, the **Sign** phase completes in just 120 s.

The paper is organised as follows. Section II reviews CRYSTALS-Dilithium and NTT basics. Section III describes the architecture, including BRAM-based memory, an interleaved dual-core Keccak architecture, an optimised rejection sampler, and a lightweight small-bit-width polynomial module. Section IV introduces a compact schedule for unifying the three security levels. Section V presents implementation and comparisons, and Section VI concludes.

II. PRELIMINARIES

A. CRYSTALS-Dilithium

CRYSTALS-Dilithium is based on the module learning-with-errors (MLWE) problem and shares the same framework

Algorithm 1 KeyGen()

```

1:  $c \leftarrow \{0, 1\}^{256}$ 
2:  $(\rho, K, \rho') \leftarrow H(c)$ 
3:  $A \leftarrow R_q^{k \times l} \leftarrow \text{ExpandA}(\rho)$ 
4:  $s_1, s_2 \in S_\eta^l, S_\eta^k \leftarrow \text{ExpandS}(K)$ 
5:  $t \leftarrow As_1 + s_2$ 
6:  $(t_1, t_0) \leftarrow \text{Power2Round}(t, d)$ 
7:  $tr \in \{0, 1\}^{256} \leftarrow H(\rho || t_1)$ 
8: return  $(pk = (\rho, t_1), sk = (\rho, \rho', tr, s_1, s_2, t_0))$ 
```

Algorithm 2 Sign(sk, M)

```

1:  $A \in R_q^{k \times l} \leftarrow \text{ExpandA}(\rho)$ 
2:  $\mu \in \{0, 1\}^{512} \leftarrow H(tr || M)$ 
3:  $\rho' \in \{0, 1\}^{512} \leftarrow H(\mu || K)$ 
4:  $\kappa \leftarrow 0, rej \leftarrow 1$ 
5: while  $rej \neq 0$  do
6:    $y \leftarrow S_{\gamma_1}^l \leftarrow \text{ExpandMask}(\rho', \kappa)$ 
7:    $\kappa \leftarrow \kappa + l$ 
8:    $w \leftarrow Ay$ 
9:    $(w_1, w_0) = \text{Decompose}(w)$ 
10:   $c \in \{0, 1\}^{256} \leftarrow H(\mu || w_1)$ 
11:   $c \in B_\tau \leftarrow \text{SampleInBall}(c)$ 
12:   $z \leftarrow y + cs_1$ 
13:   $r_0 \leftarrow w_0 - cs_2$ 
14:  if  $\|z\|_\infty < \gamma_1 - \beta$  and  $\|r_0\|_\infty < \gamma_2 - \beta$  then
15:     $h \leftarrow \text{MakeHint}_q(r_0 + ct_0, w_1)$ 
16:    if  $\|ct_0\|_\infty < \gamma_2$  and  $\sum h_i \leq \omega$  then
17:       $rej \leftarrow 0$ 
18: return  $\sigma = (z, h, c)$ 
```

as CRYSTALS-Kyber, which is currently the only standardized PQC KEM scheme. Like most lattice-based cryptographic schemes, the integer polynomial ring in CRYSTALS-Dilithium is defined as $\mathbb{Z}_q[x]/(X^n + 1)$, where q is a prime number and $X^n + 1$ represents the polynomial modulus. The prime q in CRYSTALS-Dilithium is set to $2^{23} - 2^{13} + 1$, and the polynomial degree is set to $n = 256$. The public key matrix **A** in CRYSTALS-Dilithium is defined with dimensions $k \times l$, while the secret key matrix has dimensions $l \times 1$. CRYSTALS-Dilithium includes three security levels: 2, 3, and 5. The values of q and n remain the same across all security levels, with security being increased by adjusting the values of k and l . Specifically, $k \in \{4, 6, 8\}$ and $l \in \{4, 5, 7\}$ for the three different security levels. At each security level, CRYSTALS-Dilithium consists of three main phases: key generation (**KeyGen**), signing (**Sign**), and verification (**Verify**), as shown in Algo. 1, Algo. 2, and Algo. 3, respectively. We will briefly introduce these three phases in the following, and for more details on CRYSTALS-Dilithium, the reader is kindly referred to [7].

KeyGen: In the **KeyGen** phase, the signer takes a random value c to derive ρ and K , which are used for public and secret key sampling, respectively. Then, two types of rejection sampling, $\text{ExpandA}(\rho)$ and $\text{ExpandS}(K)$, are used to generate the public key matrix **A** and the secret polynomials s_1, s_2 ,

Algorithm 3 Verify($pk, M, \sigma = (z, h, c)$)

```

1:  $A \leftarrow R_q^{k \times l} \leftarrow \text{ExpandA}(\rho)$ 
2:  $\mu \leftarrow \{0, 1\}^{512} \leftarrow H(H(\rho || t_1) || M)$ 
3:  $c' \leftarrow \text{SampleBall}(c)$ 
4:  $(w_1, w_0) \leftarrow \text{UseHint}_q(h, Az - ct_1 \cdot 2^d, 2\gamma_2)$ 
5: return  $((||z||_\infty < \gamma_1 - \beta) \text{ and } (c = H(\mu || w_1))) \text{ and } \sum h_i \leq \omega$ 

```

respectively. $\text{ExpandA}(\rho)$ and $\text{ExpandS}(K)$ utilize SHAKE-256 and SHAKE-128, respectively, to generate hash values for sampling. Next, the algorithm computes $t = As_1 + s_2$ and applies the Power2Round function to decompose the values in $\mathbb{Z}_q$ into their “high-order” and “low-bit” components, resulting in t_1 and t_2 . The hash result tr is then generated using $H(\rho || t_1)$. Finally, the public key is packed as $pk = (\rho, t_1)$ and sent to the **Verify** phase, while the secret key is packed as $sk = (\rho, K, tr, s_1, s_2, t_0)$ and sent to the **Sign** phase.

Sign: In the **Sign** phase, the public key matrix $\mathbf{A}$ and a seed ρ' are first generated. Then, this phase starts a loop that continues until a series of security conditions are met. In each loop, a commitment c is generated using $\mathbf{A}$ and a randomly sampled secret polynomial y . The values z , r_0 , and ct_0 are then checked for bounds. In addition, the counter κ is updated as $\kappa + l$ in each loop to ensure that y changes throughout the loop. Finally, the signature $\sigma = (z, h, c)$ is packed and sent to the **Verify** phase.

Verify: In the **Verify** phase, the algorithm first checks whether z and h from the **Sign** phase are within the bounds. Then, it regenerates the commitment c' . Finally, it will determine whether to accept or reject the signature.

B. NTT in CRYSTALS-Dilithium

Number Theoretic Transform (NTT) is widely used in lattice-based cryptography to accelerate polynomial multiplication. Compared with traditional algorithms such as Schoolbook algorithms, NTT reduces the computational complexity from $\mathcal{O}(n^2)$ to $\mathcal{O}(n \log n)$. Although NTT typically requires the modulus q to be “NTT-friendly” primes, i.e., satisfying $q \equiv 1 \pmod{2n}$ so that a $2n$ -th root of unity exists, mainstream lattice-based schemes such as Kyber and Dilithium are designed with this consideration and select suitable moduli accordingly. In Dilithium, a polynomial ring $\mathbb{Z}_q[X]/(X^n + 1)$ is constructed using parameters $n = 256$ and $q = 8380417$, where a $2n$ -th root of unity $\omega = 1753$ exists in the field. Let $\hat{a}$ denote the NTT of polynomial a . Then, the forward and inverse NTT (INTT) can be computed as:

$$\hat{a}_i = \sum_{j=0}^{n-1} a_j \omega^{(2i+1)j}, \quad i = 0, 1, \dots, n-1. \quad (1)$$

$$a_i = \frac{1}{n} \sum_{j=0}^{n-1} \hat{a}_j \omega^{-(2i+1)j}, \quad i = 0, 1, \dots, n-1. \quad (2)$$

C. Related Work

Several pure-hardware implementations of CRYSTALS-Dilithium have also been reported in the literature [14], [19], [20], [21], [22], [23], [24]. Ricci et al. proposed the first VHDL-based CRYSTALS-Dilithium accelerator and showed substantially higher performance on Xilinx Virtex UltraScale+ FPGAs than HLS-based designs [19]. Beckwith et al. introduced a low-latency hardware architecture targeting the third-round version of CRYSTALS-Dilithium [20]. In 2022, Zhao et al. optimised key internal modules of CRYSTALS-Dilithium, such as NTT, SampleInBall, and Decompose, achieving a $1.4\text{--}4.5\times$ reduction in hardware cost while attaining a $4.4\text{--}1.7\times$ speedup compared to previous designs [14]. In 2023, Li et al. presented an architecture based on multiple instruction parallel execution, reaching a frequency of 300 MHz on UltraScale+ FPGAs, though the design still incurred considerable area overhead [22]. In the same year, Gupta et al. proposed a lightweight CRYSTALS-Dilithium accelerator consuming only 13,975 LUTs; however, it relied on a larger amount of BRAM and exhibited significantly lower throughput than state-of-the-art designs [24]. In 2024, Zhao et al. introduced a sparse-matrix-based 2-level rejection loop design and reconstructed the polynomial module via LUTs, improving **Sign** performance but incurring an additional 9 BRAMs [21]. Most recently, Cui et al. proposed a CRYSTALS-Dilithium accelerator based on an improved Multi-Delay Commutator (MDC) NTT, achieving a 26.4% to 29.6% speed-up relative to state-of-the-art implementations, alongside a significant area increase [23]. Overall, current CRYSTALS-Dilithium accelerators still leave substantial headroom for optimisation of speed and area.

III. ARCHITECTURE OPTIMISATION PRINCIPLE

To overcome these limitations, we propose a lightweight CRYSTALS-Dilithium architecture with three key enhancements: (i) an iterative SHA-3 and sampler design that reduces area whilst improving frequency through critical-path optimisation, (ii) polynomial-module optimisations that maximise DSP utilisation and minimise input multiplexing, and (iii) elimination of the first NTT stage for small bit-width polynomials. In this section, we first present the overall CRYSTALS-Dilithium hardware architecture, then discuss its submodules in detail, including the memory configuration, SHA-3, sampling, and polynomial modules. We describe only those subfunctions that we specifically optimise in this work.

A. Overall Architecture

The overall CRYSTALS-Dilithium architecture is shown in Fig. 1. The proposed architecture consists of two parts: the signer-side and the verifier-side. The signer-side handles **Key-Gen** and **Sign** phase, while the verifier side only processes the **Verify** phase. Both sides support switching among the three security levels. At the start of the architecture, the 2-bit security-level signal is used to select the corresponding fixed parameters, which remain unchanged throughout the current execution. When the security level needs to be modified, the ongoing

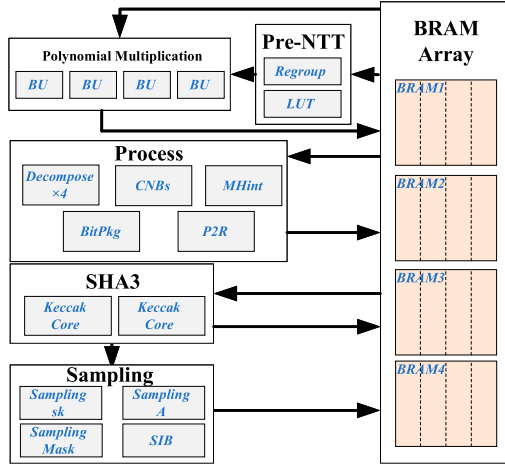


Fig. 1. The block diagram of the proposed CRYSTALS-Dilithium accelerator architecture.

operation must be reset, and the new security-level signal must be provided before restarting the computation.

In terms of memory, the proposed architecture relies on four BRAM blocks, each sized 92×1024 , to buffer polynomial results, intermediate data for processing, and the sampling unit outputs; the detailed organisation is given in Section III-B. Polynomial computations are performed by an NTT core and a small Pre-NTT unit. The NTT module supports NTT, Inverse-NTT (INTT) and point-wise multiplication (PWM). For small bit-width polynomials, the first NTT stage is handled by the Pre-NTT through LUT-based regrouping, which reduces cycles and input multiplexing. To achieve a lightweight architecture, we do not incorporate additional polynomial and memory units for pipelined 2-level rejection loops. Although such 2-level rejection loops can improve throughput, the sparse-polynomial unit in [21] incurs an overhead of 2,778 LUTs and 9 BRAMs, which is prohibitive on resource-constrained devices. In addition, the architecture includes a SHA-3 module with two Keccak cores [26] to enhance computational efficiency. The sampling module incorporates four types of samplers, two rejection sampler and SamplingMask (used to generate secret key in Sign phase), and a distributed-RAM based Sample in Ball (SIB). The processing flow from the NTT computation results to the final output is categorised as the Process module. This module includes the Decompose $\times 4$ (capable of processing four sets of data simultaneously), the Check Normal Bounds (CNBs), the Make Hint (MHint), and the Power-2-Round (P2R) module. All bit-packing (BitPkg)-related modules have been integrated, with a selector enabling flexible processing based on different polynomial types and security levels. The controller utilizes a finite state machine (FSM) to manage key modules such as SHA-3, NTT, and BRAM. To enhance flexibility, separate FSMs are used for the two BRAM ports, allowing independent control.

B. BRAM-Based Memory

The design uses BRAM as the primary on-chip memory. Since each CRYSTALS-Dilithium coefficient is 23 bits, the

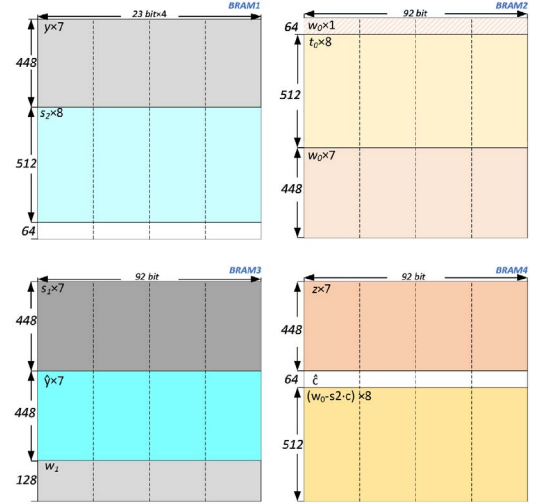


Fig. 2. The memory structure of BRAMs in the proposed accelerator.

BRAM data width is widened to 92 bits to accommodate 4 coefficients simultaneously. BRAM demand is lower during KeyGen and Verify, whilst the peak utilisation occurs in Sign. The architecture instantiates four identical BRAM blocks, each sized 92×1024 . As shown in Fig. 2, prior to executing the Sign phase, a portion of the memory space is already occupied by secret keys generated from the KeyGen phase. Specifically, t_0 is stored in BRAM2, s_1 in BRAM3, and s_2 in BRAM1. These three polynomials participate in the NTT only during the first Sign phase; thereafter their precomputed values are reused. Thus, the memory spaces stored t_0 , s_1 , and s_2 are locked to prevent overwriting in the later loops.

At the beginning of the Sign computation, the sampled signing key y is written to BRAM1. However, due to the early launch of the NTT (as described in Section IV), the ports of BRAM1 are occupied, preventing the NTT result of y ($\hat{y}$) from being written back to BRAM1. As a result, $\hat{y}$ is stored in BRAM3. The first memory block of BRAM2, with a depth of 64, serves as a buffer for the computation of $w = A \circ y$. This buffer is only overwritten by the last component of w_0 in security level 5. BRAM4 is used as a buffer for matrix A during the computation of $w = A \circ y$ and is later utilised for storing z and partial intermediate results during boundary checking in Sign phase.

C. Interleaved SHA-3 and Sampling Modules

In CRYSTALS-Dilithium, SHAKE256 from the SHA-3 family is used for sampling input data and computing hash values. Fig. 3(a) shows the baseline SHA-3/Keccak module. With a single Keccak core and a 64-bit input/output, one operation proceeds through absorb, Keccak-f[1600] permutation, and squeeze in 25, 24, and 25 cycles, respectively. Given high sampling demand of CRYSTALS-Dilithium, most implementations adopt two Keccak cores, typically in either a cascaded [14], [22] or a parallel [23]. In the cascaded scheme, two Keccak cores are cascaded as shown in Fig. 3(c), completing two rounds of Keccak-f[1600] permutation within a single cycle,

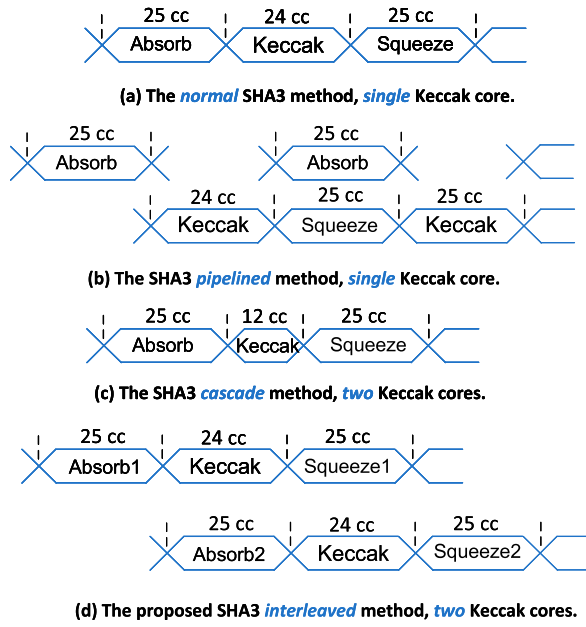


Fig. 3. SHA-3 Operation method: (a) normal method; (b) pipelined method; (c) Cascade method; (d) the proposed interleaved method.

effectively reducing the Keccak-f[1600] permutation phase to 12 cycles. However, the critical path becomes long and the maximum frequency drops. Moreover, some cascaded designs are combined with the pipelined SHA-3 architecture shown in Fig. 3(b). This pipeline separates the absorb, permutation, and squeeze phases so that the next absorb overlaps the previous squeeze. While it reduces the average cycle count, it requires an additional 1600-bit state register and extra control logic, which increases area. For example, the SHA-3 block in [14] consumes 15,784 LUTs and 4,557 FFs, while the overall design runs at only 96.9 MHz. In the parallel scheme, two Keccak cores run independently, keeping the critical path short, similar to a single core, but producing multiple hash streams concurrently; this raises area pressure due to extra buffering and requires sampler coordination [23].

To address these problems, in the proposed architecture, a novel scheduling method is first introduced for the dual-Keccak-based SHA-3 module. To prevent frequency degradation caused by cascade Keccak cores, the two Keccak cores operate independently with a fixed 26-cycle offset. As shown in Fig. 3(d), while the first Keccak core in Keccak permutation phase, the second Keccak core simultaneously enters the absorb phase, and their squeeze phases interleaved. The two SHA-3 instances operate independently, while within each SHA-3 core the absorb and squeeze phases are executed sequentially and share the same Keccak core datapath. Since most SHA-3 calls in CRYSTALS-Dilithium serve sampling, assuming each hash sampling requires four squeeze operations and the bit width of SHA-3 input/output being 64 bits, the cycle count of single sampling under the cascaded method is $25 + 12 + 25 + (12 + 25) \times 4 = 210$. By contrast, the interleaved method completes two samplings within just $25 + (24 + 25) \times 5 + 26 = 296$ cycles, i.e., an average of 148 cycles per sampling. Table I

TABLE I
AREA AND FREQUENCY COMPARISON OF DUAL-CORE SHA-3 UNDER
DIFFERENT METHODS (ARTIX-7)

Designs	Method	LUTs	FFs	Frequency (MHz)
[22]	cascade	14,971	1,635	130
[14]	cascade + pipelined	15,784	4,557	96.9
Ours	Interleaved	6,019	3,220	212.8

compares the proposed interleaved dual-core SHA-3 with state-of-the-art dual-core designs. The frequency of the proposed method closely matches that of a single Keccak core, which is nearly 2 times of cascaded method. Independent placement of the two cores eases routing congestion inside the SHA-3 block, and the overall area is lower than in [14], [22]. Relative to [14], the proposed architecture does not adopt pipelined input/output processing, resulting in significantly reduced flip-flop (FF) usage. Compared with [22], although more FFs are used, the design improves operating frequency by 63.7% while cutting LUTs by $2.5\times$. Finally, compared to fully parallel SHA-3 implementations [23], the interleaved scheme simplifies Keccak interface control and streams SHA-3 outputs directly to the sampler without intermediate buffering.

Rejection sampling in CRYSTALS-Dilithium is used to generate the public key matrix $\mathbf{A}$ and the secret vectors s_1 and s_2 (as well as y in Sign phase), which takes SHA-3 results as the input. In the proposed Samplingsk module, the 64 bits SHA-3 data block is first split into four 16-bit data blocks, each independently processed to generate four sampled results. These sets of four sampling results are then sequentially merged based on their validity, allowing the sampler to process a 64-bit SHA-3 result in a single cycle. In addition, a sampling counter triggers an output whenever the number of accepted results exceeds 15. The sampling of matrix $\mathbf{A}$ follows a similar approach. Every three 64-bit SHA-3 outputs are grouped together, generating eight sampled values over three cycles, with 2, 3, and 3 data blocks processed per cycle, respectively. Valid coefficients are accumulated through a selector, and output whenever four valid samples have been gathered. This lane-parallel design keeps the sampler aligned with the interleaved SHA-3 design and avoids intermediate buffering.

Although the proposed interleaved dual-Keccak SHA-3 architecture addresses the low frequency and reduces the need to buffer SHA-3 outputs, its interleaved outputs introduce additional challenges for the sampling module. After each sampling, the internal Buffer register and the counter (cnt) may still hold intermediate data associated with the current polynomial sampling process. When SHA-3 alternates hash blocks for two polynomials, shared state would cause interference and incorrect sampling. As shown in Fig. 4(b), in once sampling for $\mathbf{A}$, the first seventeen 64-bit hash blocks are consumed by $Poly1$. After a delay of 9 cycles, the next 17 hash blocks serve $Poly2$. Then, a further 9-cycle offset returns the stream to $Poly1$. Without proper state isolation between polynomial samplings, the use of shared buffer and counter resources would lead to erroneous output. It should be noted that Instantiating two samplers is inefficient, as rejection sampling already requires significant

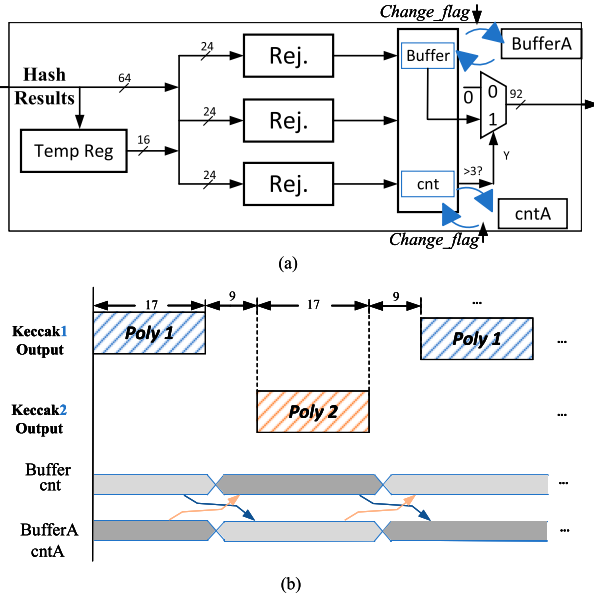


Fig. 4. Interleaved sampling mechanism, (a) the proposed interleaved sampling architecture, (b) interleaved sampling timing diagram.

hardware resources. For example, the sampler in [21] occupies over 3,293 LUTs. To address this, we therefore introduces two small registers, *BufferA* and *cntA*, in the sampler module to temporarily store the output buffer and counter values generated during the previous polynomial sampling process. As shown in Fig. 4(a), after each SHA-3 result block (consisting of either 15 or 17 output blocks, depending on the rejection sampling type) are processed, the current contents of the internal registers *Buffer* and *cnt* are exchanged with the auxiliary registers *BufferA* and *cntA*. For example, in Fig. 4(b), once the initial hash blocks for polynomial *Poly1* have been fully consumed and all valid samples are extracted, the internal state of the sampler (i.e., *Buffer* and *cnt*) is saved into the auxiliary registers *BufferA* and *cntA*. This update is triggered by the control signal *Change_flag*, which indicates that the current sampling phase is complete and that the system is ready to switch to the next polynomial. When the first hash blocks for the subsequent polynomial *Poly2* are processed, the stored values in *BufferA* and *cntA* are reloaded back into internal registers of the sampler module. At the same time, the new intermediate state generated during *Poly2* sampling is written into *BufferA* and *cntA*, preparing them for the next round of sampling switching. The addition of these two small buffers does not significantly increase hardware overhead while ensuring the smooth and correct operation of interleaved sampling.

D. NTT for Small Bit-Width Polynomials

In previous research, the Multi-path Delay Commutator-NTT (MDC-NTT) pipeline architecture [27] has been widely adopted in hardware architectures of CRYSTALS-Dilithium [14], [21], [23]. This type of NTT provides better compatibility between output throughput and memory units. However, it requires continuous input of NTT/INTT data to avoid the long pipeline

startup latency, which limits the flexibility of scheduling. In the proposed architecture, we utilise a iterative NTT design, which consists of 4 butterfly units (BUs) and FIFO-based memory, similar to [28]. The FIFO bit width is set to 92 bits, with bit depths of 32, 32, and 16, respectively. In addition, the proposed NTT design supports PWM computation and allows merging of single polynomial addition/subtraction operations with PWM computation.

Here, we further introduce two optimisation strategies to maximise the utilisation of available IP resources and to lower overall hardware cost. In addition, by leveraging a small amount of look-up tables and data regrouping, a portion of the first NTT stage is bypassed.

1) Maximising DSP Usage in Polynomial Multiplication:

The overall architecture of the proposed NTT design, consists of 4 Butterfly Units (BUs), a memory unit composed of 3 FIFOs, one signal generator (SG), and three decoders (De_BU, De_TW, and De_FIFO). Each BU includes one multiplier, one modular reducer, and one adder. The modular reduction in the proposed BU module is based on Eq. (3). This reduction module operates in a three-cycle pipeline, requiring only 94 LUTs and 109 FFs, providing a compact and efficient hardware implementation.

$$C = (c[32:23] + c[45:43] + c[42:33]) \times 2^{13} - (c[45:23] + c[45:33] + c[45:23]) + c[22:0] \quad (3)$$

In previous architectures, multipliers are typically implemented using two cascaded DSPs to construct a 23×23 -bit multiplier [14], [21], [23], which underutilises the arithmetic resources available per slice. According to the Vivado Multiplication 12.0 Handbook [29], each DSP slice contains two adders and one multiplier, allowing richer compound operations when properly configured. To improve DSP efficiency, the proposed architecture introduces a two-cycle multiplication scheme, in which each DSP is configured differently in NTT and INTT to perform optimised sub-operations. Assuming that the input of a single BU in the NTT is a and b , and the corresponding twiddle factor is ζ , the output values x and y are computed according to the Eq. (4):

$$\begin{cases} x = a + b \cdot \zeta \\ y = a - b \cdot \zeta \end{cases} \quad (4)$$

In the NTT mode, the DSP slices are configured in $A \times B + C$ computation mode, where the bit-width of A , B , and C are set as 25, 13, and 38, respectively. DSP1 and DSP2 are assigned to perform the Eq. (5), respectively:

$$\begin{cases} DSP1 = a + b \cdot \zeta[11:0] \\ DSP2 = DSP1[37:12] + b \cdot \zeta[23:12] \end{cases} \quad (5)$$

Finally, the partial results from DSP1 and DSP2 are concatenated as DSP2, DSP1[11:0], and forwarded to the modular-reduction unit, yielding the reduced value for x in Eq. (4). Compared to traditional methods, the single addition required in the computation of x is integrated into the DSP operation. In addition, by directly feeding DSP2, DSP1[11:0] into the modular reduction unit, the design eliminates the need for an extra

reduction that would otherwise be introduced by this addition, i.e., a separate subtraction step. The value of y is then computed as Eq. (6), where the $\text{Redc}()$ represents the reduction module.

$$y = 2a - \text{Redc}(\{\text{DSP2}, \text{DSP1}[11:0]\}) \quad (6)$$

During the INTT computation mode, the DSP configuration is modified to accommodate a different computational formula. It is important to note that this reconfiguration does not increase any additional area overhead. Similarly, assuming the inputs to a single BU in the polynomial module are a and b , and the twiddle factor is ζ , the outputs x_i and y_i are computed according to the Eq. (7) as follows.

$$\begin{cases} x_i = (a + b) \\ y_i = (a - b) \cdot \zeta \end{cases} \quad (7)$$

In this mode, the DSP is reconfigured to operate in the form of $(D - A) \times B + C$, where D is a 25-bit value, and the bit-widths of the other operands remain the same as in the NTT stage. Under this configuration, DSP1 and DSP2 are responsible for the following computations, respectively:

$$\begin{cases} \text{DSP1} = (a - b) \cdot \zeta[11:0] \\ \text{DSP2} = (a - b) \cdot \zeta[23:12] + \text{DSP1}[37:12] \end{cases} \quad (8)$$

At this time, the results from DSP1 and DSP2 are concatenated as DSP2, DSP1[11:0], and passed through the modular reduction unit to obtain the final result y_i . As in the NTT mode, this approach eliminates the need for one subtraction and the corresponding modular reduction. However, it is important to note that the DSP module cannot internally correct for negative values arising from the $(D - A)$ term. While the DSP can still produce a correct output, negative intermediate results are incompatible with the existing modular reduction module, leading to potential area overhead due to extra correction logic. To address this, the inputs to the D and A input of DSP in INTT mode are adjusted to $D = a + q$, and $A = b$, which ensures that the term $(D - A)$ is always positive. x_i value in Eq. (7) is computed separately using a dedicated adder. To further optimise the division by two in the INTT stage, we introduce a method inspired by [30] used in CRYSTALS-Kyber to avoid explicit division operations. First, calculate $x_i = (a + b) - q$, then, based on the sign bit of x_i (i.e., its most significant bit) and its least significant bit, we derive a correction term to obtain the properly reduced result of $x_i/2$. The final equation is given as:

$$\begin{cases} x_i[23] = 0, x_i[0] = 0 & \Rightarrow x_i[23:1] \\ x_i[23] = 0, x_i[0] = 1 & \Rightarrow x_i[23:1] + 23'h3ff001 \\ x_i[23] = 1, x_i[0] = 0 & \Rightarrow x_i[23:1] + 23'h7fe001 \\ x_i[23] = 1, x_i[0] = 1 & \Rightarrow x_i[23:1] + 23'h3ff001 \end{cases} \quad (9)$$

2) Minimise Input Multiplexing Logic: In the polynomial architecture, besides the arithmetic units such as multipliers, reducers, and adders used for computation, a significant amount of on-chip resources is also consumed by the data selection

logic for port registers of instantiated modules. For example, in previous NTT architectures, BU typically adopts a floating submodule design, in which each BU contains a complete set of multipliers, adders, and subtractors. Depending on whether the operation is NTT or INTT, different input paths are selected via multiplexers to accommodate varying computational patterns. However, this approach introduces considerable area overhead in input selection. For example, a 23-bit adder nominally requires about 23 LUTs, whereas the associated 2:1 input selection can add at least a further 46 LUTs (one 2:1 multiplexer per bit on each operand), often exceeding the cost of the adder itself.

To address this, we adopt a forward data-path in which each BU contains only forward paths and minimises input selection across internal blocks. In the proposed DSP configuration for NTT, the NTT and INTT operations are implemented using the forms $A \times B + C$ and $(D - A) \times B + C$, respectively. This keeps that the inputs to the A and B ports of the DSP identical in NTT and INTT, thereby eliminating operand switching and saving at least 46 LUTs. Moreover, For DSP2, the C port always sources the output of DSP1 in both modes, which likewise removes input multiplexing. A similar optimisation is applied at the BU outputs. According to Eq. (5) and Eq. (8), the modular reduction unit outputs x and y_i , respectively. Since results are reassembled before entering the FIFO, we order the BU outputs as (x, y) in NTT and (y_i, x_i) in INTT, avoiding an output selector altogether. In the proposed architecture, each BU module occupies only 252 LUTs and 2 DSPs, among which the modular reduction unit alone consumes 96 LUTs. In contrast, in [25], only modular multiplier within the polynomial module consumes 260 LUTs and 2 DSP slices.

In addition, the top-level control logic of the polynomial module incorporates a signal generator and three decoders, i.e., De_BU, De_TW and De_FIFO to further reduce the area overhead caused by input selection logic. The signal generator is responsible for producing twiddle factor control signals and FIFO read/write control signals. In both NTT and INTT modes, it first generates the FIFO read control and twiddle factor selection signals, and then produces other control signals through multiple shifters, aiming to minimise area consumption. The BU decoder (De_BU) is used to select the data sources for both the BU input and the twiddle factor. Its inputs include the control signals from the NTT state machine and the FIFO output control signals. For example, in the proposed architecture, single-bit of BU input receives input bits from up to four FIFO data blocks and the top-level NTT input. In addition to the data inputs, the control logic for selecting the appropriate data involves multiple control signals, including a 4-bit state signal, a 2-bit NTT/INTT/PWM mode selector, a 1-bit start signal, and a 1-bit input interleaving signal, resulting in a total of 8 control bits. For each BU, a single output bit depends on 12 to 14 input bits, depending on whether the data source contributes 4, 5, or 6 bits. A direct implementation would therefore require substantial combinational logic. We observe that, for all eight BU inputs, the selection never involves more than six data bits. This allows each BU input to be controlled by a compact 3-bit selector. By exploiting the regularity of the BU inputs in

NTT and INTT modes, we enumerate all valid combinations of the original 8 control signals and compress them into this 3-bit encoding. This approach ensures that each BU input (46-bit) is controlled by a compact 3-bit signal, thereby avoiding duplicated selection logic and reducing area overhead. Moreover, a dedicated module, *De_FIFO*, is introduced to minimise the selection logic required for the FIFO data path. Each FIFO provides a 92-bit input, which is divided into four 23-bit data blocks. For each block, a 3-bit control signal derived from the decoder determines which portion is selected. The third decoder (*De_TW*) is responsible for selecting the appropriate twiddle factors. Its inputs include the state machine, FIFO input control signals, and the NTT/INTT mode selection signal. It outputs a 5-bit control signal used to select the correct twiddle factor for either the NTT or INTT computation.

In the standard operating mode, the computation cycles for NTT, INTT, and PWM are 264 (8 pipeline set-up cycle counts and 256 operation cycle counts), 264, and 70, respectively.

3) Small Bit-width Polynomials-Based Optimisation: Notably, the range of secret key sampling results in CRYSTALS-Dilithium is relatively small. In security levels 2 and 5, the coefficient range of the secret key polynomial during the *KeyGen* phase is $[-2, 2]$, while in security level 3, the range is $[-4, 4]$. As a result, the number of possible secret key polynomial coefficients is limited: there are 5 possible values in security levels 2 and 5, and 9 possible values in security level 3. This allows for the precomputation and storage of all potential first-stage outputs in a lookup table (*LUT*), enabling the first stage of NTT computation to be skipped. For the butterfly unit (BU), when the input multiplied by the twiddle factor is zero, there is no need to precompute the result. Therefore, for a coefficient range of $[-2, 2]$, the BU produces 20 possible outputs (4×5), whereas expanding the range to $[-4, 4]$ increases the number of stored results to 72 (8×9).

However, eliminating the first stage of the NTT solely through *LUT* substitution remains challenging in practice. This is because, in FIFO-based NTT designs, when the inputs follow the pairing sequence defined by the NTT (e.g., (0, 128)), the output of the BU is first stored in a FIFO until its corresponding pair (e.g., (64, 192)) is processed, in order to generate the input pair (0, 64) for the second stage. Therefore, if the inputs are still fed into the NTT module following the original pairing pattern ((0,128), (1,129)...), then the outputs obtained from the *LUT* must still be reordered via the FIFO. In this case, using *LUT* only reduces the pipeline startup latency by eliminating the initial computation cycles of the first-stage BUs. To address this issue, if the *LUT* module is capable of directly outputting data in the pairing order required by the second stage of the NTT, the first stage can be effectively eliminated. In the proposed architecture, this is achieved by regrouping the small bit-width polynomial coefficients after the *LUT* generation phase, such that their output naturally conforms to the second-stage input pairing pattern. This enables, for the first time, the NTT computation in CRYSTALS-Dilithium hardware designs to begin directly from the second stage. This regrouping operation is performed on-the-fly during sampling and has no impact on the NTT cycle count.

In CRYSTALS-Dilithium, the data at order 0 in the second stage originates from the first stage computation result of pairing (0, 128), while the data at order 64 in second stage comes from the first-stage result of pairing (64, 196). In a LUT-based NTT design, the output at index 0 is generated in the first cycle, whereas the value at index 64 appears only after sixteen cycles, so the first NTT stage cannot be eliminated. In addition, a dual-port BRAM has only two access ports, and the indices (0, 128, 64, and 192) map to four distinct memory locations that cannot be read in one cycle. To overcome this limitation, the sampling results are regrouped before entering the LUT so that the required inputs can be obtained using only the two BRAM ports. Fig. 5 shows the process of data regrouping in the proposed design. The module shifts 16 sampled data (64-bits each in total) by four cycles, then regroupes data blocks before storing them separately. For example, in the first cycle of regrouping, data blocks with order (64, ..., 71) is merged with (0, ..., 7) and stored at position 0 in BRAM (the first address of the current NTT input), while data blocks with order (72, ..., 79) is combined with (8, ..., 15) and stored at position 1. Similarly, data blocks with order (128, ..., 135) is merged with (192, ..., 199) and stored at position 8. In the proposed design, each pair of data blocks fetched from BRAM in a single access is sufficient to provide all the necessary inputs for a second-stage NTT butterfly computation. For example, upon initiating the NTT, the BRAM reads from addresses 0 and 8, retrieving polynomial coefficients with indices (0, ..., 7, 64, ..., 71) and (128, ..., 135, 192, ..., 199), respectively. From these, the input set (0, 64, 128, 192) can be directly selected and routed through *LUT* to supply the necessary operands for the second-stage computation as shown in Fig. 6(a). In each cycle of the second-stage computation, only one of the paired outputs from the first stage is required. This implies that, during the NTT input phase, each memory address must be accessed twice in order to obtain both outputs of input pairs in first stage for the second-stage pairing. As shown in Fig. 6(b), skipping the first NTT stage leaves only seven stages, each requiring 32 cycles. In addition, the LUT introduces two extra pipeline-initialization cycles for data preparation, while its own computation completes in a single cycle. With this approach, the total NTT computation time is reduced to 235 cycles, resulting in an 11.7% speed improvement. It is important to note that the additional hardware introduced by this approach remains within acceptable limits. The regrouping module requires only 104 LUTs, while the *LUT* overhead is determined by the number of BUs in the current polynomial module. Each BU is allocated 84 LUTs for data selection purposes. This optimisation is applicable not only to CRYSTALS-Dilithium but also to other lattice-based cryptographic schemes that employ small-bit-width NTTs, such as CRYSTALS-Kyber.

IV. COMPACT SCHEDULING

Benefiting from the iterative NTT architecture, the proposed architecture is more flexible in terms of scheduling. In polynomial matrix computation, all the processes required to compute it are completed for a particular row of the matrix, and the result

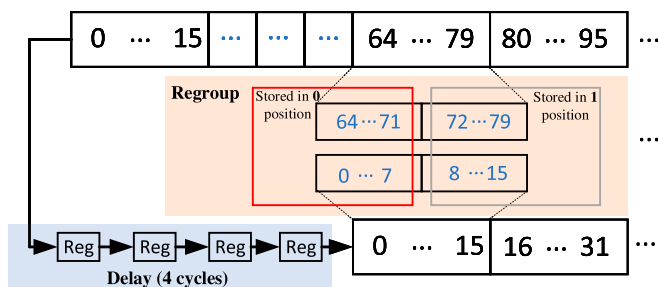
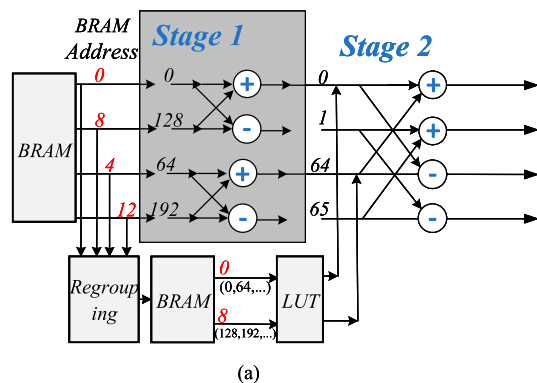


Figure 1 consists of two block diagrams, (a) and (b), illustrating the proposed architecture.

(a) Encoder: This diagram shows a sequence of operations. It starts with a zero input (0) entering a PWM block. The output of this block is y, which is then added to the output of BRAM2. The result is fed into a series of PWM blocks, each receiving inputs from BRAM3 and BRAM4. The output of the final PWM block is A, which is added to the output of BRAM2. The result is then fed into an INTT block. The output of the INTT block is a sequence of w0, w1, and BRAM3.

(b) Decoder: This diagram shows a single PWM block followed by an INTT block. The PWM block receives inputs from BRAM3, s2, and c. Its output is y, which is added to the output of BRAM4. The result is then fed into the INTT block, which outputs z.



(a)

$\times 8$

BU Startup 8 40 72 264 cc

BU Output

(b)

$\times 7$

LUT BU Startup 1 11 43 75 235 cc

BU Output

is stored afterwards. As shown in Fig. 7(a), when computing $t = A \circ y$, the process first calculates $A[i][j] \cdot y[j]$ and iterates l times. During each iteration, one port of BRAM4 retrieves $A[i][j]$ (while the other port collects the sampled values of A at the same time), BRAM3 provides the corresponding y value, and the computed PWM result is stored in BRAM2 at addresses 0 to 63. As the iteration progresses k times, the addresses of BRAM4 and BRAM3 increment sequentially, while BRAM2 only stores data in the same address. In addition, the maximum memory of matrix $\mathbf{A}$ in BRAM4 is selected according to different security levels to handle data buffering. For example, in security levels 2 and 3, the maximum depth of matrix $\mathbf{A}$ in BRAM4 is 9×64 , and the counter for reading the components in $\mathbf{A}$ loops within the range of $[0, 8]$. Finally, the PWM results stored in BRAM2 are extracted and sent to the INTT and Decompose computation to obtain $w_0[i]$ and $w_1[i]$. This

This scheduling strategy is widely applied in the proposed architecture. For example, in Fig. 7(b), the PWM computation in the proposed NTT architecture supports simultaneous polynomial addition when computing $z = \hat{y} + c \cdot s_1[i]$. The architecture performs $z[i] = \hat{y}[i] + c \cdot s_2[i]$ in sequence, then performs the INTT computation, and subsequently iterates the process k times. Since each iteration can be treated as a complete computational process, this means that the intermediate computation result $z[i]$ in each of the k iterations can be stored in the same address, as they are no longer needed in subsequent iterations. The storage location for intermediate results is flexible. For example, in Fig. 7(a), this part of the data is stored in BRAM2. However, when computing z , BRAM2 is already filled with data, thus, BRAM4 is used as the memory location for intermediate variables. All intermediate results can be reused in a single polynomial store, thus avoiding the problem of adding 6 additional BRAMs and additional polynomial multiplication modules as mentioned in [14].

Some targeted scheduling optimisation measures can also contribute to speed improvement. At the beginning of the **KeyGen** and **Sign** phases, the first Keccak core waits until the second core has fully completed its output before restarting and proceeding to the next computation. This is because the NTT input stage only requires 32 cycles, meaning that the interval between the generation of two key polynomials is sufficient to complete a single NTT input. Compared to waiting until all key sampling is completed before running NTT, this optimisation allows NTT computation to start about 380 cycles earlier at security level 5, thereby improving overall performance. In addition, adjustments have been made to the sampling sequence based on the varying number of operations contained in each polynomial computation iteration. For example, in the **KeyGen** phase, the sampling of $\mathbf{A}$ and s_2 is interleaved to ensure that the remaining components of $\mathbf{A}$ do not exceed the maximum capacity of buffer space in BRAM4. In the **Verify** phase, since only matrix $\mathbf{A}$ is needed to be sampled, each Keccak core in

TABLE II
RESOURCE CONSUMPTION IN PROPOSED SIGNER-SIDE ARCHITECTURE

Module	LUTs	FFs	DSPs	BRAMs
SHA-3	6,019	3,220	0	0
Polynomial Multi.	2,601	2,730	8	0
Sampling	4,035	1,157	0	0
Process	2,328	2,185	0	0
Memory	0	0	0	12
Total	20,623	13,854	8	12

the SHA-3 module runs $k/2$ times per iteration (corresponding to the three different security levels). The SHA-3 module then pauses execution, waiting for the NTT module to complete its computations. Once the PWM calculations for the current iteration are finished, the SHA-3 module resumes operation.

V. IMPLEMENTATION RESULTS AND COMPARISON

In this section, we report the resource utilisation and end-to-end performance of the proposed CRYSTALS-Dilithium architecture, and compare it with state-of-the-art designs. The design was simulated, synthesised and implemented on an Artix-7 (XC7A200T) device using AMD Xilinx Vivado 2020.1 *Design Suite*. All secret-key dependent operations, including rejection sampling and NTT-based polynomial arithmetic, are implemented in constant time. The sampler always processes a fixed number of SHA-3 outputs and the polynomial multiplication and Pre-NTT modules use access patterns determined only by public parameters, so for a given security level the cycle count is independent of the secret key and sampled values, providing inherent resistance to timing attacks.

A. Resource Utilisation

Table II summarises the resource utilisation of the proposed architecture. The signer-side consumes 20,623 LUTs, 13,854 FFs, 8 DSPs and 12 BRAMs in total. The SHA-3 block containing two Keccak cores occupies 6,019 LUTs and 3,220 FFs in total. Specifically, the two Keccak cores use 3,150 and 2,869 LUTs, respectively, and each consumes 1,610 FFs. Compared with [14] (15,784 LUTs) and [21] (8,110 LUTs), this represents a reduction of approximately $2.6\times$ and $1.3\times$, respectively, whilst maintaining comparable computational speed. Although the SHA-3 block employs an interleaved-output scheme that increases sampler control complexity, the complete sampling module (Table II) still uses only 4,035 LUTs including different samplers required by CRYSTALS-Dilithium, which is comparable to the sampler in [21]. The NTT core (polynomial multiplication) requires 2,601 LUTs, 2,730 FFs and 8 DSPs, and supports NTT, INTT, PWM and second-stage startup. The Pre-NTT module includes regrouping and LUT modules, consuming 104 LUTs and 336 LUTs respectively. The on-chip memory footprint is limited to 12 BRAMs. Post-implementation power for the signer-side is estimated at 0.879 W by Vivado.

B. Overall Performance and Comparison

Table III compares area and timing with state-of-the-art designs. Several works accelerate the Sign rejection loop by adding dedicated compute and memory; for example, [21] provisions an extra 2,778 LUTs, 615 FFs and 9 BRAMs for the z and h stages to shorten runtime. To remain lightweight, our design does not include the two-level rejection loop or these additional resources. Moreover, our optimisation strategies do not conflict with the 2-level rejection loop in [21]. If necessary, the proposed architecture can also increase these additional resources to enhance the speed and efficiency in the rejection loop phase. Therefore, although Table III reports both the best-case and average signing performance, the comparison of Sign speed focuses only on the best-case results in order to highlight the advantage of the proposed architecture.

Direct area comparison is difficult because designs employ different optimisation strategies and make varying use of LUTs, FFs, DSPs and BRAMs. Following [30], we therefore report the *equivalent number of slices* (ENS) by uniformly converting resources: every four LUTs or eight FFs count as one slice, one DSP counts as 100 slices, and one BRAM counts as 200 slices. ENS figures for all architectures are listed in Table III. The proposed design uses only 10,088 ENS, making it among the most lightweight CRYSTALS-Dilithium accelerators reported to date. Relative to the previously most compact design [24], our architecture reduces ENS by 14.2% whilst supporting all three security levels (whereas [24] targets only level 5). For a fairer speed comparison, we also synthesised our design on a speed-grade -2 device, achieving 178.6 MHz. At the same speed grade, our latencies in security level 5 are 59.4 s, 143.3 s and 69.4 s for **KeyGen**, **Sign** and **Verify**, respectively, which correspond to $6.5\times$, $4.9\times$ and $6.0\times$ improvements over [24]. Compared with [14], our design reduces overall area by 15.9%. At the lowest security level, **KeyGen** latency in cycles is comparable (4,144 cycles), while **Sign** and **Verify** require more cycles because the iterative SHA-3 behaves as a single-core SHA-3 outside the sampling phases. Nevertheless, the iterative SHA-3 shortens the critical path and raises frequency, yielding significant end-to-end speed-ups: at the lowest level, **KeyGen**, **Sign** and **Verify** improve by 54.5%, 33.1% and 43.2%, respectively; at level 5, the corresponding gains are 44.7%, 26.2% and 37.0%. Design in [14] did not share the speed grade level of its FPGA. Nevertheless, even under the assumption of a -2 speed level, the proposed architecture still demonstrates superior performance in speed.

The architecture of [22] utilizes more DSPs to increase the speed and thus has more ENS (14,497), which is an increase of 30.4% compared to our proposed architecture. However, the proposed architecture using only 4 BUs of NTT has better performance in both **KeyGen** and **Verify** phases compared to [22] in terms of cycles count and frequency. In addition, in the **Sign** phase, the proposed architecture improves the speedup by 24.1%, 26.5%, and 22.6% in the three security levels, respectively. Compared to architecture in [25], the proposed architecture uses the unified design to support three security levels, and the overall area consumption is even lower than the hardware

TABLE III
POST-PAR IMPLEMENTATION RESULTS FOR OUR CRYSTALS-DILITHIUM ARCHITECTURE (AREA AND TIMING) AND COMPARISON WITH THE STATE-OF-THE-ART DESIGNS (ARTIX-7)

Design	Security Level	Cycle counts (k)	F. (MHz)	Time(s) Key/Sign(B) ¹ /Sign(A) ¹ /Verify	Area				
					LUTs	FFs	DSPs	BRAMs	ENS ²
[14]	2	4.2/8.3/31.6/4.4		43.3/85.6/326.1/45.4					
	3	5.9/11.4/49.5/6.1	96.9	60.9/117.5/510.8/63.0	29,998	10,366	10	11	11,995
	5	8.8/15.8/55.3/9.0		90.1/163.0/570.7/92.6					
[25] ³	2	7.8/12.2/52.0/7.7		49/76.7/327/48	18,558	7,342	10	17	9957
	3	13.0/17.5/89.2/11.2	159	82/110.1/561/ 71	19,614	8,466	10	21	11,162
	5	20.2/24.3/93.7/15.9		127/152.8/589.3/100	20,973	9,677	10	28	13,053
[22]	2	4.4/9.8/30.1/5.0		33.9/75.4/231.5/38.5					
	3	7.3/14.8/49.5/8.0	130	56.1/113.8/380.7/61.6	32,320	9,734	24	14	14,497
	5	10.9/20.2/55.2/12.7		83.8/155.4/424.6/97.6					
[24]	5	63.2/113.9/-/67.9	163	387.7/698.8/-/416.6	13,975	6,875	4	35	11,753
[21]	2	3.9/-/12.8/4.2		32.8/-/107.6 /35.3					
	3	6.5/-/21.4 /6.9	119	54.6/-/179.8/58.0	30,304	13,559	16	28	16,470
	5	10.9/14.9/27.6/11.5		91.6/126.0/231.9/96.6					
[23]	2	4.8/-/28.8/5.2		25.1/-/150.1 /27.2					
	3	7.9/-/48.6/8.5	191	41.4/-/254.5 /44.5	52,888	31,213	38	11	23,124
	5	11.8/-/54.7/12.5		61.8/-/286.4 /65.4					
Ours	2	4.2/12.2/41.4/5.5		19.7/57.3/194.5/25.8					
	3	6.4/17.8/68.6/8.2	212.8	30.1/83.6/322.4/38.5	20,623	13,854	8	12	10,088
	5	10.6/25.6/82.9/12.4		49.8/120.3/389.3/58.3					

1. Sign(B): Signing cycle counts in the best-case scenario; Sign(A): Signing cycle counts in the average scenario.

2. ENS (equivalent number of slices) = $LUT/4 + FF/8 + DSP \times 100 + BRAM \times 200$ [30] 3. Zynq-7000.

area of its security level 3 architecture. Meanwhile, in terms of speed, the proposed architecture achieves better performance in terms of both frequency and cycles.

Compared to the architecture in [21] that includes a rejection loop, the proposed architecture reduces hardware area by 38.7%. Even after removing the additional modules for sparse matrix multiplication and the BRAM modules for the rejection loop in [21], the proposed architecture still achieves a 27.4% reduction in hardware area (13,899 ENS in [21]). In terms of speed, for the KeyGen and Verify phases, the proposed architecture improves performance by 39.9-45.6% and 32.9-39.6% at all three security levels, respectively, compared to [21]. For the Sign phase, the design in [21] requires 126 s at security level 5. In contrast, the proposed architecture completes the process in just 120.3 s. Notably, this is achieved despite using fewer resources. The proposed architecture utilizes only half of the BUs in the NTT compared to the design in [21]. Part of the best-case Sign time in [21] is attributable to its two-level design, which adds independent polynomial-multiplication modules to increase parallelism and throughput. Relative to [23], our design reduces hardware by 2.3× yet still achieves 11.3% to 27.3% speed-ups in KeyGen and 5.0% to 13.5% in Verify across all three security levels. In short, the proposed architecture is more lightweight whilst delivering higher speed.

VI. CONCLUSION

In this paper, we propose the most lightweight CRYSTALS-Dilithium hardware architecture to date. First,

an iterative dual-Keccak SHA-3 architecture is presented, where two Keccak cores operate independently with a 26-cycle offset, producing interleaved hash outputs. This ensures a unified critical path with the NTT and other modules while reducing resource overhead from cascading Keccak cores and input/output pipelining. Additionally, the rejection sampler is optimised by introducing two small registers to store intermediate values and counters, reducing resource consumption, to accommodate the interleaved SHA-3 output. Second, lookup tables and a data regrouping method are utilised in the architecture to fully eliminate the first stage of NTT computation for low-bit-width secret polynomials in CRYSTALS-Dilithium, achieving an 11.7% reduction in NTT computation cycles. In terms of scheduling, a more compact method is employed to split and iteratively process polynomial computations on a row-by-row basis. The intermediate memory required for all polynomial computations is only the same amount of space as a single polynomial. Experimental results show that compared to the most lightweight CRYSTALS-Dilithium hardware architecture, the proposed design reduces hardware area by 14.2%. Additionally, at all three security levels, the KeyGen and Verify phases achieve speed improvements of 27.3% and 13.5%, respectively. Specifically, at security level 5, the best-case Sign phase takes only 120.3 s, compared to 126 s in state-of-the-art designs. The architecture meets the performance requirements of resource-constrained environments while enabling further speed improvements through the integration of additional polynomial computation modules and a 2-level rejection loop.

REFERENCES

- [1] R. L. Rivest, A. Shamir, and L. Adleman, "A method for obtaining digital signatures and public-key cryptosystems," *Commun. ACM*, vol. 21, no. 2, pp. 120–126, 1978.
- [2] V. S. Miller, "Use of elliptic curves in cryptography," in *Proc. Conf. Theory Appl. Cryptographic Techn.*, New York, NY, USA: Springer-Verlag, 1985, pp. 417–426.
- [3] P. W. Shor, "Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer," *SIAM Rev.*, vol. 41, no. 2, pp. 303–332, 1999.
- [4] L. K. Grover, "A fast quantum mechanical algorithm for database search," in *Proc. 28th ACM Symp. Theory Comput.*, 1996, pp. 212–219.
- [5] D. Moody et al., "Status report on the first round of the NIST post-quantum cryptography standardization process," Nat. Inst. of Standards and Technol., *Tech. rep.*, 2019. [Online]. Available: <https://www.nist.gov/publications/status-report-firstround-nist-post-quantum-cryptography-standardization-process>
- [6] R. Avanzi et al., "CRYSTALS-Kyber algorithm specifications and supporting documentation," *NIST Post-Quantum Cryptography Project, 3rd Round Candidate Algorithms*, 2022. [Online]. Available: <https://www.pq-crystals.org/kyber/data/kyber-specificationround3-20210804.pdf>
- [7] V. Lyubashevsky et al., "CRYSTALS-Dilithium," *NIST Post-Quantum Cryptography Project, Third Round Candidate Algorithms*, 2022. [Online]. Available: <https://csrc.nist.gov/CSRC/media/Presentations/crystalsdilithium-round-3-presentation/images-media/session-1-crystals-dilithiumlyubashevsky.pdf>
- [8] D. J. Bernstein, A. Hülsing, S. Kölbl, R. Niederhagen, J. Rijneveld, and P. Schwabe, "The SPHINCS+ signature framework," in *Proc. ACM SIGSAC Conf. Comput. Commun. Secur.*, 2019, pp. 2129–2146.
- [9] T. Prest et al., "Falcon," *NIST Post-Quantum Cryptography Project, 3rd Round Candidate Algorithms*, 2022. [Online]. Available: https://post-quantum.nl/slides/PQC4_Slides_Thomas.Prest.pdf
- [10] C. A. Melchor et al., "Hamming quasi-cyclic (HQC)," *NIST PQC Round*, vol. 2, no. 4, p. 13, 2018.
- [11] "FIPS 203: Module-lattice-based key-encapsulation mechanism standard," Gaithersburg, MD, USA: NIST, 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.203.ipd.pdf>
- [12] "FIPS 204: Module-lattice-based digital signature standard," Gaithersburg, MD, USA: NIST, 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.204.ipd.pdf>
- [13] "FIPS 205: Stateless hash-based digital signature standard," Gaithersburg, MD, USA: NIST, 2023. [Online]. Available: <https://nvlpubs.nist.gov/nistpubs/FIPS/NIST.FIPS.205.ipd.pdf>
- [14] C. Zhao et al., "A compact and high-performance hardware architecture for CRYSTALS-Dilithium," *IACR Trans. Cryptographic Hardware Embedded Syst.*, vol. 2022, no. 1, pp. 270–295, 2022.
- [15] Y. Cui, J. Chen, Z. Ni, Z. Zhang, C. Wang, and W. Liu, "Instruction-based high-performance hardware controller of CRYSTALS-Kyber with balanced resource utilization," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 5, pp. 2394–2407, May 2025.
- [16] P. Nannipieri, S. Di Matteo, L. Zulberti, F. Albicocchi, S. Saponara, and L. Fanucci, "A RISC-V post quantum cryptography instruction set extension for number theoretic transform to speed-up CRYSTALS algorithms," *IEEE Access*, vol. 9, pp. 150798–150808, 2021.
- [17] L. Li, Q. Tian, G. Qin, S. Chen, and W. Wang, "Compact instruction set extensions for Dilithium," *ACM Trans. Embed. Comput. Syst.*, vol. 23, pp. 1–21, Mar. 2024.
- [18] X. Ji et al., "ECO-CRYSTALS: Efficient cryptography CRYSTALS on standard RISC-V ISA," *IEEE Trans. Comput.*, vol. 74, no. 2, pp. 401–413, Feb. 2025.
- [19] S. Ricci et al., "Implementing CRYSTALS-Dilithium signature scheme on FPGAS," in *Proc. 16th Int. Conf. Availability, Rel. Secur.*, 2021, pp. 1–11.
- [20] L. Beckwith, D. T. Nguyen, and K. Gaj, "High-performance hardware implementation of CRYSTALS-Dilithium," in *Proc. Int. Conf. Field-Programmable Technol. (ICFPT)*, Piscataway, NJ, USA: IEEE Press, 2021, pp. 1–10.
- [21] H. Zhao, C. Zhao, W. Zhu, B. Yang, S. Wei, and L. Liu, "Sparse polynomial multiplication-based high-performance hardware implementation for CRYSTALS-Dilithium," in *Proc. IEEE Int. Symp. Hardware Oriented Secur. Trust (HOST)*, 2024, pp. 150–159.
- [22] X. Li, J. Lu, D. Liu, A. Li, S. Yang, and T. Huang, "A high speed post-quantum crypto-processor for crystals-Dilithium," *IEEE Trans. Circuits Syst. II, Exp. Briefs*, vol. 71, no. 1, pp. 435–439, Jan. 2024.
- [23] Y. Cui, J. Zhong, B. Wang, T. Xu, C. Wang, and W. Liu, "High-performance hardware implementation of Crystals-Dilithium based on improved MDC-NTT," *IEEE Trans. Comput.*, vol. 74, no. 9, pp. 2896–2908, Sep. 2025.
- [24] N. Gupta, A. Jati, A. Chattopadhyay, and G. Jha, "Lightweight hardware accelerator for post-quantum digital signature CRYSTALS-Dilithium," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 8, pp. 3234–3243, Aug. 2023.
- [25] T. Wang, C. Zhang, P. Cao, and D. Gu, "Efficient implementation of Dilithium signature scheme on FPGA SoC platform," *IEEE Trans. Very Large Scale Integr. (VLSI) Syst.*, vol. 30, no. 9, pp. 1158–1171, Sep. 2022.
- [26] K. Team, "Keccak in VHDL," Oct. 2020. [Online]. Available: <https://keccak.team/hardware.html>
- [27] S. He and M. Torkelson, "A new approach to pipeline FFT processor," in *Proc. Int. Conf. Parallel Process.*, Piscataway, NJ, USA: IEEE Press, 1996, pp. 766–770.
- [28] Z. Ni, A. Khalid, W. Liu, and M. O'Neill, "Towards a lightweight CRYSTALS-Kyber in FPGAs: An ultra-lightweight BRAM-free NTT core," in *Proc. IEEE Int. Symp. Circuits Syst. (ISCAS)*, 2023, pp. 1–5.
- [29] Xilinx, "7 Series DSP48E1 slice user guide," UG479 (v1.10), Mar. 27, 2018.
- [30] W. Guo and S. Li, "Highly-efficient hardware architecture for CRYSTALS-Kyber with a novel conflict-free memory access pattern," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 70, no. 11, pp. 4505–4515, Nov. 2023.



Ziying Ni (Member, IEEE) received the Ph.D. degree in electronics and electrical engineering from Queen's University Belfast (QUB) Belfast, U.K., in 2024 under an EPSRC Studentship, and the M.S. degree in electronics and communication engineering from Nanjing University of Aeronautics and Astronautics (NUAA), Nanjing, China, in 2021. He is currently a Research Fellow with the Centre for Secure Information Technologies (CSIT) of QUB, Belfast, U.K. His research interests include hardware security, and cryptographic engineering.



side channel attacks, and

cryptographic hardware.

Ayesha Khalid (Senior Member, IEEE) received the B.E. and M.S. degrees from the National University of Sciences and Technology (NUST, Pakistan) and the University of Engineering and Technology (UET-Taxila, Pakistan), respectively. She is currently a Lecturer with Queen's University Belfast, NI, U.K. She won a DAAD Scholarship Award for the Ph.D. (2015) from RWTH Aachen, Germany. She is a member of VAS Technical Committee of IEEE ISCAS. Her research interests include lattice-based cryptography, embedded systems security, side channel attacks, and cryptographic hardware.



Zhaoyu Zhang received the B.E. and M.S. degrees in automation from the University of Science and Technology of China, Hefei, China, in 2017 and 2020, respectively. He is currently working toward the Ph.D. degree in computer science with Queen's University Belfast (QUB), Belfast, U.K. His research interests include artificial intelligence, computer vision, machine learning and hardware security.



Yijun Cui (Member, IEEE) received the B.Sc. degree in information engineering and the Ph.D. degree in information and communication systems from Nanjing University of Aeronautics and Astronautics, Nanjing, China, in 2010 and 2020, respectively. He was a visiting Ph.D. student with the Data Security System Group in the Centre of Secure Information Technologies, Queen's University Belfast, U.K., from 2014 to 2015. He is currently an Associate Professor with the College of Electronics and Information Engineering of NUAA.

His research interest includes hardware security.



Weiqiang Liu (Senior Member, IEEE) received the B.Sc. degree in information engineering from Nanjing University of Aeronautics and Astronautics (NUAA), China, and the Ph.D. degree in electronic engineering from Queens University Belfast, U.K., in 2006 and 2012, respectively. He is currently a Professor and the Vice Dean of the College of Electronic and Information Engineering, NUAA. He has published two research books and over 200 leading journal and conference papers. He has been awarded the prestigious Excellent Young Scholar Award by

NSFC in 2020. He serves as the Associate Editor for IEEE TRANSACTIONS ON COMPUTERS, IEEE TRANSACTIONS ON CIRCUITS AND SYSTEMS I: REGULAR PAPERS, and IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING, the Steering Committee Member of TVLSI, the Guest Editor of Proceedings of IEEE. He is the program co-chair of IEEE ARITH 2020 and NANOARCH 2022, and also TPC Member for DAC, DATE, ARITH et al. His research interests include energy efficient and secure computing circuits and systems.



Máire O'Neill (Senior Member, IEEE) received the M.Eng. degree with distinction and the Ph.D. degree in electrical and electronic engineering from Queen's University Belfast (QUB), in 1999 and 2002, respectively. She is currently a Regius Professor in electronics and computer engineering, the Director of Institute of Electronics, Communications and Information Technology (ECIT) and the Director of UK Research Institute in Secure Hardware and Embedded Systems (RISE). She has authored two research books and has over 190

international peer-reviewed conference and journal publications. She was previously an Associate Editor for IEEE TRANSACTIONS ON COMPUTERS and IEEE TRANSACTIONS ON EMERGING TOPICS IN COMPUTING and Guest Editor for several journals. She has been a Technical Program Committee Member for many international conferences, including DAC, CHES, DATE et al. She is a fellow of Royal Academy of Engineering, fellow of Irish Academy of Engineering, and member of the Royal Irish Academy.